```
pip install transformers datasets evaluate seqeval torch -qqq
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.6/43.6 kB 2.0 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 4.3 MB/s eta 0:00:00
  Building wheel for seqeval (setup.py) ... done
```

```
import datasets
import transformers

datasets.disable_progress_bar()
transformers.utils.logging.disable_progress_bar()
```

```
from tqdm import tqdm

for i in tqdm(range(100), disable=True):
    pass
```

Dataset Selection, Data Preprocessing & Model Setup

```
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForTokenClassification,
    DataCollatorForTokenClassification,
    TrainingArguments,
    Trainer
)
import evaluate
import numpy as np

from datasets import load_dataset
```

```
from datasets import load_dataset

# Load the Dataset
dataset=load_dataset("lhoestq/conll2003")
chunk_labels=[
    "O", "B-ADJP", "I-ADJP", "B-ADVP", "I-ADVP", "B-CONJP", "I-CONJP",
    "B-INTJ", "I-INTJ", "B-LST", "I-LST", "B-NP", "I-NP", "B-PP", "I-PP",
    "B-PRT", "I-PRT", "B-SBAR", "I-SBAR", "B-UCP", "I-UCP", "B-VP", "I-VP"
]

id2label={i: label for i, label in enumerate(chunk_labels)}
label2id={label: i for i, label in enumerate(chunk_labels)}
print("Labels successfully mapped!")

# Load Tokenizer & DistilBERT
model_checkpoint="distilbert-base-cased"
tokenizer=AutoTokenizer.from_pretrained(model_checkpoint)
model=AutoModelForTokenClassification.from_pretrained(
    model_checkpoint,
    num_labels=len(chunk_labels),
    id2label=id2label,
    label2id=label2id
)

# Data Preprocessing
def tokenize_and_align_labels(examples):
    tokenized_inputs=tokenizer(
        examples["tokens"], truncation=True, is_split_into_words=True
    )
    labels=[]
    for i, label in enumerate(examples["chunk_tags"]):
        word_ids=tokenized_inputs.word_ids(batch_index=i)
        previous_word_idx=None
        label_ids=[]
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:
```

```
                label_ids.append(label[word_idx])
            else:
                label_ids.append(-100)
            previous_word_idx=word_idx
        labels.append(label_ids)

    tokenized_inputs["labels"]=labels
    return tokenized_inputs
tokenized_datasets=dataset.map(tokenize_and_align_labels, batched=True)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens),
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
Labels successfully mapped!
DistilBertForTokenClassification LOAD REPORT from: distilbert-base-cased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING       :those params were newly initialized because missing from the checkpoint. Consider training on your dowr
```

Training & Evaluation

```
# Load the seqeval metric
```

```
seqeval=evaluate.load("seqeval")

def compute_metrics(p):
    predictions, labels = p
    predictions=np.argmax(predictions, axis=2)
    true_predictions=[
        [chunk_labels[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    true_labels=[
        [chunk_labels[l] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    results=seqeval.compute(predictions=true_predictions, references=true_labels)
    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"],
    }
data_collator=DataCollatorForTokenClassification(tokenizer=tokenizer)

args=TrainingArguments(
    output_dir="./distilbert-chunking",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
)

trainer=Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
```

```
        data_collator=data_collator,
        compute_metrics=compute_metrics,
        processing_class=tokenizer,
    )
    trainer.train()
```

```
Downloading builder script:        6.34k/?  [00:00<00:00,  82.5kB/s]
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as
  super().__init__(loader)
```

[2363/2634 2:33:31 < 17:37, 0.26 it/s, Epoch 2.69/3]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	0.414837	0.183803	0.913257	0.902825	0.908011	0.951618
2	0.147287	0.159725	0.920344	0.914908	0.917618	0.956914

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as
  super().__init__(loader)
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWarning: Precision and F-score are ill-
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as
  super().__init__(loader)
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWarning: Precision and F-score are ill-
  _warn_prf(average, modifier, msg_start, len(result))
```

[2634/2634 2:54:47, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	0.414837	0.183803	0.913257	0.902825	0.908011	0.951618
2	0.147287	0.159725	0.920344	0.914908	0.917618	0.956914
3	0.117264	0.157145	0.922527	0.917007	0.919758	0.958101

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as
  super().__init__(loader)
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWarning: Precision and F-score are ill-
  _warn_prf(average, modifier, msg_start, len(result))
TrainOutput(global_step=2634, training_loss=0.19418702020551004, metrics={'train_runtime': 10502.3172,
'train_samples_per_second': 4.011, 'train_steps_per_second': 0.251, 'total_flos': 525452463338040.0, 'train_loss':
0.19418702020551004, 'epoch': 3.0})
```

Inference

```
from transformers import pipeline
chunking_pipeline=pipeline("token-classification", model=model, tokenizer=tokenizer, aggregation_strategy="simple")

text="John works at Google in California"
predictions=chunking_pipeline(text)

print("Input:", text)
for entity in predictions:
    print(f"Word/Phrase: {entity['word']} | Tag: {entity['entity_group']} | Score: {entity['score']:.4f}")
```

```
Input: John works at Google in California
Word/Phrase: John | Tag: NP | Score: 0.9989
Word/Phrase: works | Tag: VP | Score: 0.8720
Word/Phrase: at | Tag: PP | Score: 0.9981
Word/Phrase: Google | Tag: NP | Score: 0.9978
Word/Phrase: in | Tag: PP | Score: 0.9987
Word/Phrase: California | Tag: NP | Score: 0.9987
```

Comparison (POS vs Chunking)

POS Tagging (Word-Level Tagging): It involves assigning grammatical labels to each individual word. (Example: John – Noun, works – Verb). This task is less complex for machines since it requires only a small context window to assign POS tags.

Chunking (Phrase-level grouping): Here, words that are related are grouped together into phrases (e.g., “The quick brown fox” = Noun Phrase). This task has a medium level of difficulty since, apart from recognizing word types, the model should also recognize phrases’ boundaries using the BIO tag set.

Report

The Core Difference

Even though both tasks fall under token classification, they differ greatly when it comes to their purposes:

In POS tagging, one deals with high detail: "What grammatical function does this particular word play?" This approach focuses on individual tokens, where 'Google' becomes a Proper Noun and 'in' a Preposition.

In chunking, one deals with structure: "How do words combine into meaning units?" This task focuses on spans of text: it sees "Google in California" as being part of phrase construction with the help of BIO (beginning, inside, outside) tags for marking up phrase boundaries.

Challenges faced: The Tokenization Labeling Problem

The major difficulty that comes up in utilizing BERT-type models for token labeling is related to the mismatch between the word in question and the tokens generated through the tokenization process carried out by the model. The model uses sub-word tokenization, such as WordPiece. For instance, "tokenization" can be tokenized into "token", "##ize", and "##tion". If there is only